

Module Documentation: AAS Generation and Submodel Mapping Subsystem

1. Overview

The module enables the upload of **KBL or VEC files** and generates an **Asset Administration Shell (AAS)** with one or more generated **Submodels** as as described in [SAS 7.3](#) and in the [BaSyx WebUI issue](#).

In addition, a product image can optionally be uploaded as a thumbnail.

The module consists of the Vue component `UploadAASKblVec.vue` and several helper modules in the `KblVecUtils` folder.

2. Purpose

The module is intended to extract structured information from KBL/VEC XML files, present it to the user in a selectable form, and then transfer it into a repository as an AAS structure.

Core functions:

- Upload and validation of KBL/VEC files
- Extraction of data points from XML
- Display of data points as a selectable tree
- Automatic prepopulation of required AAS fields
- Generation of Submodels and SubmodelElements
- Upload of Submodels, attachments, and an optional product image
- Creation and linking of the AAS shell

3. Responsibilities

The module is responsible for:

Area	Description
File upload	Receiving and validating KBL/VEC files
XML processing	Extraction, normalization, and type derivation from XML data
User interaction	Displaying extracted data points as a selectable tree
Field inference	Deriving required AAS fields such as ShortID, AAS ID, Asset ID, Name, Description, and Asset Kind
Submodel generation	Creating Submodels from selected data points
Backend communication	Uploading AAS, Submodels, SubmodelElements, attachments, and thumbnails
Error support	Generating helpful hints from backend responses

The module is not primarily responsible for:

Non-responsibility	Reason
Domain validation of all KBL/VEC contents	The module extracts and converts data, but it does not replace complete domain validation
Persistence logic in the backend	It uses backend endpoints but does not implement their storage behavior
General AAS management	It creates and links an AAS from upload data, but it is not a complete AAS management module

4. Position in the System

The central UI component is:

```
src/components/UploadAASKblVec.vue
```

This component uses helper modules from:

```
src/utlis/KblVecUtils
```

These helper modules handle specialized tasks such as XML conversion, data-point tree generation, selection logic, field inference, and Submodel generation.

5. Main Component: UploadAASKblVec.vue

Purpose

`UploadAASKblVec.vue` is a dialog component. It allows users to upload KBL or VEC files and generate an Asset Administration Shell from them.

Props

Name	Type	Description
<code>modelValue</code>	<code>boolean</code>	Controls whether the dialog is open or closed

Emits

Event	Description
<code>update:modelValue</code>	Triggered when the visibility of the dialog changes

Main Tasks

The component:

1. validates uploaded KBL/VEC files,
2. progressively extracts data points from XML,
3. displays these data points in a selectable tree,
4. allows editing of required AAS fields,
5. generates Submodels from the user selection,
6. uploads Submodels, SubmodelElements, and attachments,
7. creates the AAS shell,
8. links Submodels to the shell,
9. optionally uploads a product image as a thumbnail.

6. Workflow

The functional workflow of the module is:

1. The user opens the upload dialog.
2. The user selects a KBL or VEC file.
3. Optionally, the user selects a product image.
4. The file is validated.
5. XML data is extracted progressively.
6. Extracted data points are displayed as selectable items.
7. Required AAS fields are inferred automatically.
8. The user reviews and edits the AAS fields.
9. The user selects data points for export.
10. The module generates Submodels from the selection.
11. Submodels and SubmodelElements are uploaded.
12. Attachments are uploaded.
13. The AAS shell is created.
14. Submodels are linked to the AAS shell.
15. Optionally, the product image is uploaded as a thumbnail.

7. Helper Modules

7.1 KblVecXmlConversionUtils.ts

Purpose:

Provides helper functions for XML/JSON conversion, type derivation, and AAS-compliant identifiers.

Important functions:

Function	Task
<code>isXmlObject</code>	Checks whether a value can be treated as an XML object
<code>getXmlElementEntries</code>	Extracts XML element entries
<code>getAttributeEntries</code>	Extracts attributes
<code>extractXmlId</code>	Determines XML IDs
<code>buildAttachmentSmePath</code>	Builds paths for attachment SubmodelElements
<code>toSubmodelElementIdShort</code>	Generates AAS-compliant <code>idShort</code> values
<code>inferTypedValue</code>	Derives types from XML values
<code>normalizeAssetKind</code>	Normalizes Asset Kind values

7.2 KblVecUploadUtils.ts

Purpose:

Bundles upload, lookup, and error-hint logic.

Important functions:

Function	Task
<code>normalizeForLookup</code>	Normalizes strings for search and comparison operations
<code>tokenizePath</code>	Splits paths into tokens
<code>getElementPath</code>	Determines the path of an element
<code>stringifyUnknown</code>	Robustly converts unknown values to strings
<code>splitIdAndSuffix</code>	Splits an ID and suffix
<code>extractBackendMessages</code>	Extracts messages from backend responses
<code>buildBackendSpecificHint</code>	Builds user-friendly backend hints

7.3 KblVecDataPointTreeUtils.ts

Purpose:

Creates a visible tree of data points from an XML element.

Important exports:

Export	Task
<code>ExtractedDataPoint</code>	Structure for extracted data points
<code>DataPointTreeNode</code>	Tree node structure
<code>buildDataPointTree</code>	Creates the data-point tree
<code>appendVisibleTreeRows</code>	Generates visible rows for the UI
<code>collectExportPointsFromSelectedNodes</code>	Collects data points from selected nodes
<code>collectSelectableNodeKeys</code>	Collects selectable node keys
<code>getSelectableKeysForRow</code>	Determines selectable keys per row
<code>countSelectableNodes</code>	Counts selectable nodes

7.4 KblVecSelectionStateUtils.ts

Purpose:

Manages selection states within the data-point tree.

Important functions:

Function	Task
<code>toggleSelection</code>	Toggles a selection
<code>toggleRowSelection</code>	Toggles the selection of a tree row
<code>isRowFullySelected</code>	Checks whether a row is fully selected
<code>isRowPartiallySelected</code>	Checks whether a row is partially selected

7.5 KblVecRequiredFieldUtils.ts

Purpose:

Automatically derives required AAS fields from data points or XML elements.

Relevant AAS fields:

Field	Meaning
<code>ShortID</code>	Short identifier of the AAS
<code>AAS ID</code>	Unique AAS ID
<code>Asset ID</code>	Unique Asset ID
<code>Name</code>	Display name
<code>Description</code>	Description
<code>Asset Kind</code>	Type of asset

Important functions:

Function	Task
<code>createEmptyRequiredFieldValues</code>	Creates empty default values
<code>inferRequiredFieldValues</code>	Infers field values from existing data

7.6 KblVecSubmodelGenerationUtils.ts

Purpose:

Generates Submodels from selected data points.

Important functions:

Function	Task
<code>buildSubmodelsFromSelection</code>	Creates Submodels from the user selection
<code>buildHandoverDocumentationSubmodel</code>	Creates a Submodel for handover documentation

7.7 KblVecSubmodelElementUtils.ts

Purpose:

Provides helper functions for creating SubmodelElements and ensuring unique `idShort` values.

Important functions:

Function	Task
<code>ensureUniqueIdShortForCollection</code>	Ensures unique <code>idShort</code> values within a collection
<code>addTypedPropertyToCollection</code>	Adds typed Properties to a collection

8. Data Flow

```
graph TD
    A[KBL/VEC file] --> B[UploadAASKblVec.vue]
    B --> C[XML parsing and data-point extraction]
    C --> D[KblVecDataPointTreeUtils]
    D --> E[Selectable data-point tree]
    E --> F[KblVecSelectionStateUtils]
    F --> G[Selected data points]
    G --> H[KblVecRequiredFieldUtils]
    H --> I[Required AAS fields]
    I --> J[KblVecSubmodelGenerationUtils]
    J --> K[Submodels]
    K --> L[KblVecSubmodelElementUtils]
    L --> M[SubmodelElements]
    M --> N[Backend upload]
    N --> O[AAS shell + Submodel linking + attachments + thumbnail]
```

9. Important Technical Notes

Performance

The component extracts data points progressively. The chunk sizes within the component should be chosen to achieve a good balance between performance and UI responsiveness.

Backend Error Hints

When backend interactions are changed, care should be taken to ensure that the hints generated by `KblVecUploadUtils.buildBackendSpecificHint` remain understandable and helpful for users.

10. Maintenance Notes

The following points should be considered when extending the module:

Change	Affected Areas
New export formats	<code>KblVecRequiredFieldUtils</code> , <code>KblVecSubmodelGenerationUtils</code>
New field inference rules	<code>KblVecRequiredFieldUtils</code>
Changes to XML normalization	<code>KblVecXmlConversionUtils</code>
Changes to the selection UI	<code>KblVecDataPointTreeUtils</code> , <code>KblVecSelectionStateUtils</code>
Changes to backend upload	<code>UploadAASKblVec.vue</code> , <code>KblVecUploadUtils</code>
Changes to the Submodel structure	<code>KblVecSubmodelGenerationUtils</code> , <code>KblVecSubmodelElementUtils</code>

11. Known Risks

Risk	Impact	Recommendation
Large XML files	May affect UI performance	Review progressive extraction and chunk sizes
Backend response formats change	Error hints may become incomplete	Adapt <code>extractBackendMessages</code> and <code>buildBackendSpecificHint</code>
Inaccurate field inference	Incorrect AAS metadata may result	Keep required fields manually editable
Ambiguous XML structures	Submodel generation may become ambiguous	Add tests with realistic KBL/VEC files